

A Lightweight Edge-AI Dual-Model IMU Framework for Fall and Crash-Like Event Detection with Automated SOS Alerting

Devesh Nath Jha¹, Vishwajit Kumar Kanth¹, Kumar Saurav¹
Raja Dey^{1,2}, Prof. (Dr.) G. S. Mitra Thakur¹

¹Department of Computer Science & Engineering (AI&ML),
Dr. B. C. Roy Engineering College, Durgapur, West Bengal – 713206, India.

²*Corresponding Author (Project Guide)*

Author Emails: deveshjnv2002@gmail.com, vishwajitkumarkanth@gmail.com,
saurav.ambasta777@gmail.com, rajadeycse@gmail.com,
gour.mitrathakur@bcrec.ac.in

Abstract

Road traffic accidents claim approximately 1.35 million lives annually, with a significant proportion of fatalities attributable to delayed emergency response. This paper presents a real-time, edge-deployable inference framework that monitors both human passenger falls and high-impact vehicular crash-like events using the 6-axis Inertial Measurement Unit (IMU) available in any modern smartphone.

The core contribution is a *Dual-Model Architecture* comprising **two independently trained binary** 1D Convolutional Neural Network (CNN) models sharing a single hot-swappable inference engine. These are *not* a single unified multi-class model; each is a separate binary classifier serving a different operating context. The fall detection model was trained and evaluated on the **MobiFall v2.0** benchmark dataset. The crash detection model was trained and evaluated exclusively on a **physics-derived synthetic** vehicle crash dataset; it has *not* been validated on real-world crash recordings.

Each model is compressed from 137.54 KB to 15.38 KB using TensorFlow Lite (TFLite) dynamic range quantization, an 88.8% reduction. On MobiFall v2.0, the

1D-CNN achieved 96.65% accuracy with 8,481 trainable parameters and a system-level inference latency of approximately 2 ms. Crash-like event detection on the synthetic dataset achieved 93.61% accuracy; these results represent simulation-based feasibility evidence only.

Keywords: fall detection, crash-like event detection, edge AI, TFLite, 1D-CNN, inertial measurement unit, IoT, passenger safety, real-time inference, MobiFall, SOS alerting, sensor fusion, TinyML

Contents

1	Introduction	4
2	Related Work	5
2.1	Fall Detection Using Wearable Sensors	5
2.2	Vehicle Crash Detection	5
2.3	Edge-Deployed Neural Networks for Safety Systems	5
2.4	Gap in the Literature	6
3	Datasets	6
3.1	Dataset Summary	6
3.2	MobiFall v2.0: Human Fall Dataset	6
3.3	Synthetic Vehicle Crash Dataset	7
4	System Architecture	8
4.1	Data and Preprocessing Layer	8
4.2	Training and Quantization Layer	9
4.3	Inference Engine and Mode Switching	9
4.4	API and Alert Layer	9
4.5	Data Privacy and User Consent	10
5	Deep Learning Model Architectures and Training	10
5.1	Architecture Designs	10
5.2	Training Protocol	11
6	Experimental Results	12
6.1	Performance Metrics	12
6.2	Fall Detection Results — MobiFall v2.0	13
6.3	Crash-Like Event Detection Results — Synthetic Dataset	13
6.4	Latency Disambiguation	14
6.5	System-Level Performance	15
6.6	Inference Engine Behaviour	15
6.7	Comparison with State of the Art (Fall Detection Only)	16
7	Edge Deployment and Practical Considerations	17
7.1	TFLite Conversion	17
7.2	Sensor Orientation and Coordinate Invariance	18
7.3	Limitations	18
8	Conclusion	19

1 Introduction

Every year, approximately 1.35 million people lose their lives in road traffic crashes [World Health Organization, 2018]. Survival depends heavily on response speed. Research shows that mortality increases substantially when the interval between a crash and first medical contact exceeds 60 minutes — the *Golden Hour* [Cowley, 1976].

Modern vehicles have active safety systems such as airbags and Advanced Driver-Assistance Systems (ADAS), but these do not automatically contact emergency services or next-of-kin after a crash. Smartphones present an underexplored opportunity: virtually every passenger already carries one, and its 6-axis Inertial Measurement Unit (IMU) can record the violent inertial signature of a collision or human fall. What has been missing is a lightweight, on-device software layer that acts on this data reliably and instantly without routing any raw sensor data to a remote server.

Core Architectural Clarification

This system uses **two independently trained binary classifiers**, not a single unified model distinguishing normal motion, falls, and crashes:

- **Human mode** — fall detection: classifies each 2.56-second IMU window as *Normal Activity* or *Fall Detected*.
- **Vehicle mode** — crash detection: classifies each window as *Normal Driving* or *Crash-Like Event*.

In the current implementation, mode switching is **manual**: the user selects the operating mode via the mobile dashboard. Automatic context-aware switching has not been implemented and is left as future work.

Contributions

1. A **Dual-Model 1D-CNN framework**: two separately trained binary classifiers sharing one hot-swappable runtime inference engine.
2. A **Stateful Ring-Buffer Inference Engine** (*FallDetectionEngine*) with system-level latency under 3 ms and runtime model hot-swapping.
3. A **Physics-Based Synthetic Crash Dataset**: 200 crash and 200 normal-driving sequences from first-principles kinematic models.
4. A **Real-Time WebSocket Pipeline** (FastAPI Application Programming Interface / Uvicorn) sending Save Our Souls (SOS) SMS via Twilio with Global Positioning System (GPS) coordinates upon crash detection.

The paper is organized as follows: Section 2 surveys prior work; Section 3 describes both datasets; Section 4 covers system architecture; Section 5 details model design and training; Section 6 presents experimental results; Section 7 addresses deployment considerations; Section 8 concludes.

2 Related Work

2.1 Fall Detection Using Wearable Sensors

Threshold-based approaches dominated early fall detection literature. [Kangas et al. \[2008\]](#) showed that peak acceleration alone could flag falls with sensitivity above 95%, though frequent false alarms from Activities of Daily Living (ADL) such as jumping or sitting sharply limited practical use.

Deep learning changed this picture. [Musci et al. \[2020\]](#) replaced hand-tuned rules with Long Short-Term Memory (LSTM) networks, reporting near-perfect sensitivity. [Nho et al. \[2021\]](#) achieved 99.1% on SisFall using a bidirectional LSTM. The recurrent nature of LSTMs, however, limits throughput on low-power edge hardware since each timestep must be processed sequentially.

Convolutional methods offer a parallelizable alternative. [Wang et al. \[2020\]](#) demonstrated that a 1D-CNN applied to a 50 Hz accelerometer stream could capture fall-characterising temporal patterns with far fewer sequential operations. The present work builds on this line, applying TFLite quantization to achieve a 15.38 KB model footprint at sub-3 ms system-level latency.

2.2 Vehicle Crash Detection

Most crash detection research targets dedicated hardware such as on-board Event Data Recorders (EDRs). [Bhatt et al. \[2017\]](#) proposed a rule-based detector using accelerometer magnitude thresholds (92% accuracy), though hard braking and potholes trigger similar signatures. [Jiang et al. \[2020\]](#) used machine learning on naturalistic driving data but required hardware in instrumented test vehicles. Recent work applying deep learning to smartphone IMUs shows promise [[Al-Turjman et al., 2024](#)], though labelled real-world crash recordings remain scarce. This work circumvents the data problem via physics-based synthesis.

2.3 Edge-Deployed Neural Networks for Safety Systems

TFLite now supports weight quantization, pruning, and operator fusion making deep models viable on microcontrollers [[David et al., 2021a](#)]. Surveys on Tiny Machine Learning (TinyML) confirm that safety-critical inference on IoT devices is a practical possibil-

ity [Abadade et al., 2023]. David et al. [2021b] demonstrated ARM Cortex-M processors running models in only 20 KB.

2.4 Gap in the Literature

To the best of our current survey, few works jointly consider human falls and vehicular crash-like events within a single inference framework, and fewer still integrate automated emergency alerting with GPS coordinates into the pipeline. This work addresses that gap while explicitly acknowledging that crash detection has only been validated on synthetic data.

3 Datasets

3.1 Dataset Summary

Table 1 provides a concise summary of both datasets.

Table 1: Summary of Datasets Used in This Work

Property	MobiFall v2.0	Synthetic Crash Dataset
Source	Vavoulas et al. [2014]	Physics-based generator (this work)
Sensor type	6-axis IMU (accel. + gyro.)	6-axis IMU (simulated)
Sampling rate	87 Hz (resampled to 50 Hz)	50 Hz (generated directly)
Subjects / sequences	24 human subjects	200 crash + 200 normal sequences
Number of classes	2 (Fall / Normal ADL)	2 (Crash / Normal Driving)
Window size	128 samples (2.56 s at 50 Hz)	128 samples (2.56 s at 50 Hz)
Window overlap	50% (step = 64 samples)	50% (step = 64 samples)
Availability	Publicly available	Generated by authors

3.2 MobiFall v2.0: Human Fall Dataset

The MobiFall v2.0 dataset [Vavoulas et al., 2014] was collected using a Samsung Galaxy S3 smartphone in the trouser pocket of 24 subjects (aged 22–47; 19 male, 5 female) at 87 Hz, capturing 6-axis IMU data.

- **Fall classes (label = 1):** Forward fall (FOL), Fall on knee (FKL), Backward stumble (BSC), Sideways lateral fall (SDL).
- **ADL classes (label = 0):** Standing (STD), Walking (WAL), Jogging (JOG), Jumping (JUM), Stairs up (STU), Stairs down (SCH), Car step-in (CSI), Car step-out (CSO).

Raw files are parsed by extracting rows with exactly 6 numeric values, resampled to 50 Hz, and segmented into 128-sample windows at 50% overlap (Section 4.1).

3.3 Synthetic Vehicle Crash Dataset

Real labelled crash datasets are nearly unavailable for academic use. We designed a physics-based generator modelling two driving states.

Normal Driving Model

- Z-axis gravity: $a_z \approx 9.8 \pm 0.5 \text{ m/s}^2$
- Longitudinal accel.: $a_y \sim \mathcal{N}(0, 1.0) \text{ m/s}^2$
- Lateral accel.: $a_x \sim \mathcal{N}(0, 0.5) \text{ m/s}^2$
- Yaw rotation: $\omega_z \sim 30 \text{ deg/s}$ (sinusoidal)

Crash Event Model (three phases)

1. **Pre-crash:** 50–100 samples of normal driving.
2. **Impact (5–12 samples, ≈ 100 –240 ms at 50 Hz):**

$$a_y^{\text{crash}} \sim \mathcal{N}(-300, 50) \text{ m/s}^2 \approx 30.6 \text{ g} \quad (1)$$

$$\omega_{x,y,z}^{\text{crash}} \sim \mathcal{N}(300, 100) \text{ deg/s} \quad (2)$$

3. **Post-crash rest:** Gravitational vector redistributed across axes at random orientation; gyroscopes return to near-zero:

$$\begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}_{\text{rest}} \sim \mathcal{N}(\hat{n} \cdot 9.8, 0.1) \quad (3)$$

The generator produced 400 sequences (200 crash + 200 normal), each 300–500 samples long, processed through the same sliding-window pipeline.

Physical Validity and Explicit Limitation

NHTSA NCAP frontal barrier crash tests at 35 mph report peak decelerations of 20–40 g [National Highway Traffic Safety Administration (NHTSA), 2023]. Our synthetic spike of $\approx 30.6 \text{ g}$ falls within this range. The *impact followed by near-silence* pattern is used by commercial EDRs as the primary crash indicator [Bhatt et al., 2017].

The synthetic dataset does not capture oblique impacts, rollovers, or multi-vehicle collisions. All crash detection results are simulation-based feasibility evidence and must not be interpreted as real-world crash detection capability.

4 System Architecture

The system has four layers (Figure 1): (1) Data & Preprocessing, (2) Training & Quantization, (3) Inference Engine, (4) API & Alert.

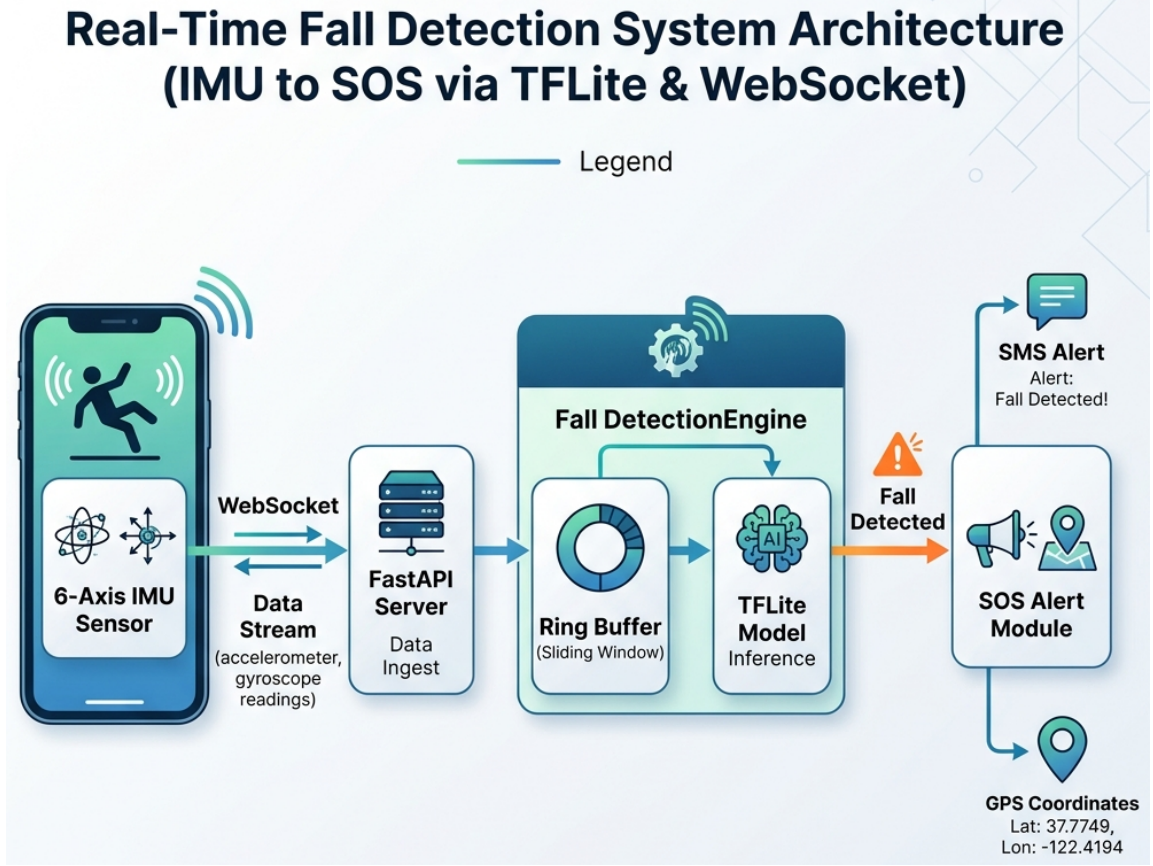


Figure 1: System architecture. Raw 6-axis IMU data streams at 50 Hz via WebSocket to the FastAPI server. The *FallDetectionEngine* accumulates samples in a ring buffer and triggers TFLite inference every 64 samples. Upon crash detection, an emergency SMS with GPS coordinates is dispatched via Twilio to pre-configured emergency contacts.

4.1 Data and Preprocessing Layer

Windows of size $W = 128$ are extracted from the time-series $\mathbf{X} \in \mathbb{R}^{T \times 6}$ with step $S = 64$ (50% overlap):

$$\mathbf{W}_i = \mathbf{X}[i \cdot S : i \cdot S + W, :] \quad i = 0, 1, \dots, \left\lfloor \frac{T - W}{S} \right\rfloor \quad (4)$$

At 50 Hz, each window spans 2.56 seconds. Browser sensor data (HTML5 DeviceMotionEvent API) is resampled to 50 Hz via linear interpolation:

$$\mathbf{x}(t'_k) = \mathbf{x}_i + \frac{t'_k - t_i}{t_{i+1} - t_i}(\mathbf{x}_{i+1} - \mathbf{x}_i), \quad t'_k = t_0 + k \cdot 20 \text{ ms} \quad (5)$$

4.2 Training and Quantization Layer

Keras .h5 models are converted to TFLite using dynamic range quantization (float32 weights $\rightarrow$ 8-bit integers):

$$\hat{w} = \text{round}\left(\frac{w}{s}\right), \quad s = \frac{\max(|w|)}{2^{b-1} - 1}, \quad b = 8 \quad (6)$$

4.3 Inference Engine and Mode Switching

The *FallDetectionEngine* has three design properties:

1. **Ring Buffer:** Thread-safe deque (maxlen $W = 128$). Inference triggers every $S = 64$ new samples.
2. **Hot-Swap Capability:** Pre-loaded TFLite interpreters keyed by mode ("human"/"car") are switched via `set_mode()`, which also clears the buffer. **Mode switching is currently manual** (dashboard toggle); automatic context detection has not been implemented.
3. **Static Guardrail:** Output is overridden to Normal Activity when the acceleration magnitude standard deviation falls below threshold:

$$\sigma_{\text{acc}} = \text{std} \left(\left\| \begin{bmatrix} a_x(t) \\ a_y(t) \\ a_z(t) \end{bmatrix} \right\|_2, \quad t = 1 \dots W \right) < \sigma_{\text{static}} = 0.05 \quad (7)$$

The threshold $\sigma_{\text{static}} = 0.05$ was determined **empirically**: a stationary phone produces $\sigma_{\text{acc}} \approx 0.02$, while normal walking produces $\sigma_{\text{acc}} \approx 0.20\text{--}0.28$. The value 0.05 provides clear separation between these two regimes and is not asserted to be universally valid across all device placements or activity types.

4.4 API and Alert Layer

Two interfaces are exposed:

- **WebSocket (/ws/predict):** Accepts JSON sensor samples at up to 50 Hz; returns prediction events.

- **REST (POST /predict/single):** HTTP single-sample evaluation for testing.

Upon crash detection (confidence > 0.5), an emergency SMS is sent via Twilio to **pre-configured contacts** defined in the system environment file. **The system does not integrate with official emergency dispatch services** such as 112 or 911. If Twilio credentials are absent, alerts are logged locally in mock mode.

4.5 Data Privacy and User Consent

- **No raw data storage:** IMU readings are processed in-memory within the ring buffer and never written to disk or sent to any third-party server other than Twilio.
- **GPS for SOS only:** Location data is used solely for the emergency SMS payload; it is not logged or used for any other purpose.
- **Explicit consent required:** The mobile dashboard requests browser permissions for DeviceMotionEvent and Geolocation before streaming begins. No data is transmitted without user consent.

5 Deep Learning Model Architectures and Training

5.1 Architecture Designs

Four architectures were evaluated under identical conditions. All accept input shape (128,6) and output a sigmoid probability. Both the fall model and crash model use the same 1D-CNN architecture, **each trained independently on its respective dataset.**

1D-CNN (Selected Architecture)

$$z_k(t) = \text{ReLU} \left(b_k + \sum_{j=1}^6 \sum_{\tau=0}^{K-1} w_{jk}(\tau) \cdot x_j(t - \tau) \right) \quad (8)$$

Topology: Conv1D(64,k=3,ReLU) → MaxPool(2) → Dropout(0.3) → Conv1D(32,k=3,ReLU) → GlobalMaxPool → Dense(32,ReLU) → Dense(1,Sigmoid). **Total trainable parameters: 8,481.**

LSTM

LSTM(64) → Dropout(0.4) → Dense(32) → Dense(1). Total parameters: 20,289.

CNN-LSTM Hybrid

Conv1D(64) → MaxPool → Dropout(0.3) → LSTM(64) → Dropout(0.3) → Dense(32) → Dense(1). Total parameters: 36,353.

SIMPLE 1D-CNN ARCHITECTURE FOR IMU TIME-SERIES

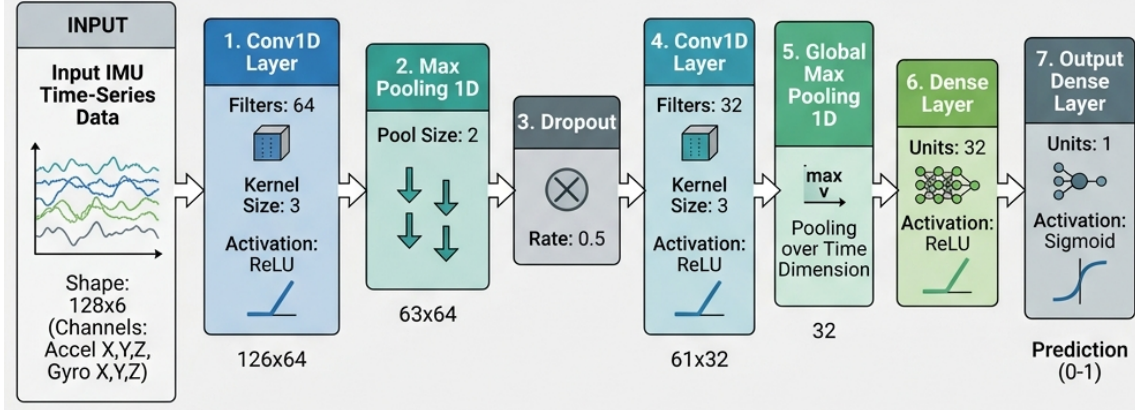


Figure 2: 1D-CNN architecture used for both fall and crash models, each trained independently on its respective dataset.

Lightweight Transformer

Multi-head attention (2 heads, key-dim 32) $\rightarrow$ GlobalAveragePooling $\rightarrow$ Dense(32) $\rightarrow$ Dense(1). Total parameters: 1,557.

5.2 Training Protocol

- **Optimizer:** Adam ($\eta = 0.001$)
- **Loss:** Binary cross-entropy
- **Batch size:** 64
- **Max epochs:** 50 with early stopping
- **Early stopping:** Monitor `val_loss`, patience=5, restore best weights. Training terminates before 50 epochs when validation loss stops improving. The training curves in Figure 3 were produced under this configuration.
- **Random seed:** 42

Split Protocols

Two distinct protocols are used, one per dataset:

- **MobiFall v2.0 — Subject-wise split (fall model):** Subject IDs are extracted from MobiFall filenames. Subjects are partitioned into disjoint training (80%) and test (20%) cohorts. No subject appears in both sets, preventing data leakage from shared intra-subject patterns. This is the *only* evaluation protocol for the fall model.
- **Synthetic crash dataset — Stratified window-level split (crash model):** Synthetic sequences have no subject IDs. A stratified window-level split is used (80/20, `stratify=y`, seed 42). Class proportions are preserved across splits. This is the *only* evaluation protocol for the crash model.

These two protocols are not interchangeable. Their results are reported separately in Section 6.

Loss and Decision Functions

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] \quad (9)$$

$$\hat{y}_i = \mathbf{1}[\hat{p}_i \geq \tau], \quad \tau = 0.5 \quad (10)$$

6 Experimental Results

6.1 Performance Metrics

Let TP, TN, FP, FN denote true positives, true negatives, false positives, and false negatives for the positive class (Fall or Crash):

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (11)$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (12)$$

$$\text{Recall (Sensitivity)} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (13)$$

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (14)$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}} \quad (15)$$

Specificity and Precision are distinct metrics. Specificity (true negative rate) is not the same as Precision (positive predictive value), and both terms are used with their correct definitions throughout.

6.2 Fall Detection Results — MobiFall v2.0

Table 2 reports test-set performance on MobiFall v2.0 under the subject-wise split protocol. These results apply exclusively to the fall detection task.

Table 2: Architecture Comparison on MobiFall v2.0 (Subject-wise split, 20% test set). Precision, Recall, and F1-Score are for the Fall class.

Model	Acc.	Prec.	Recall	F1	Params	TFLite
1D-CNN	96.65%	95.0%	99.0%	97.0%	8,481	15.38 KB
CNN-LSTM	97.04%	95.5%	99.1%	97.2%	36,353	15.38 KB
LSTM	96.06%	94.8%	98.7%	96.7%	20,289	15.38 KB
Transformer	95.90%	94.5%	98.4%	96.4%	1,557	15.38 KB

Note on uniform TFLite sizes: All four variants produce 15.38 KB after quantization because all use the same 1D-CNN definition with 8,481 parameters. Dynamic range quantization applies a per-layer float32-to-int8 compression; identical architecture yields identical quantized sizes. The original Keras .h5 files are 137.54 KB each, giving a compression ratio of 88.8%.

Key fall detection metrics for the selected 1D-CNN:

- **Recall (Sensitivity):** 99.0% — 99 of 100 fall events detected.
- **Precision:** 95.0% — 95% of flagged windows are genuine falls.
- **F1-Score:** 97.0%

6.3 Crash-Like Event Detection Results — Synthetic Dataset

Disclaimer: The crash detection model has not yet been validated on real vehicle crash recordings. The results in this section should be interpreted as **simulation-based feasibility evidence only**. The synthetic dataset produced highly separable classes, leading to high performance; this should not be interpreted as evidence of real-world crash generalization.

Table 3 reports crash detector performance on the synthetic dataset (stratified window-level split). These results are entirely independent of the MobiFall fall detection results above.

5-Fold Cross-Validation on the Synthetic Crash Dataset

To assess stability of the crash model, a 5-fold stratified cross-validation was performed using the full 2,000-window **synthetic crash dataset** only. This result does not apply to fall detection on MobiFall.

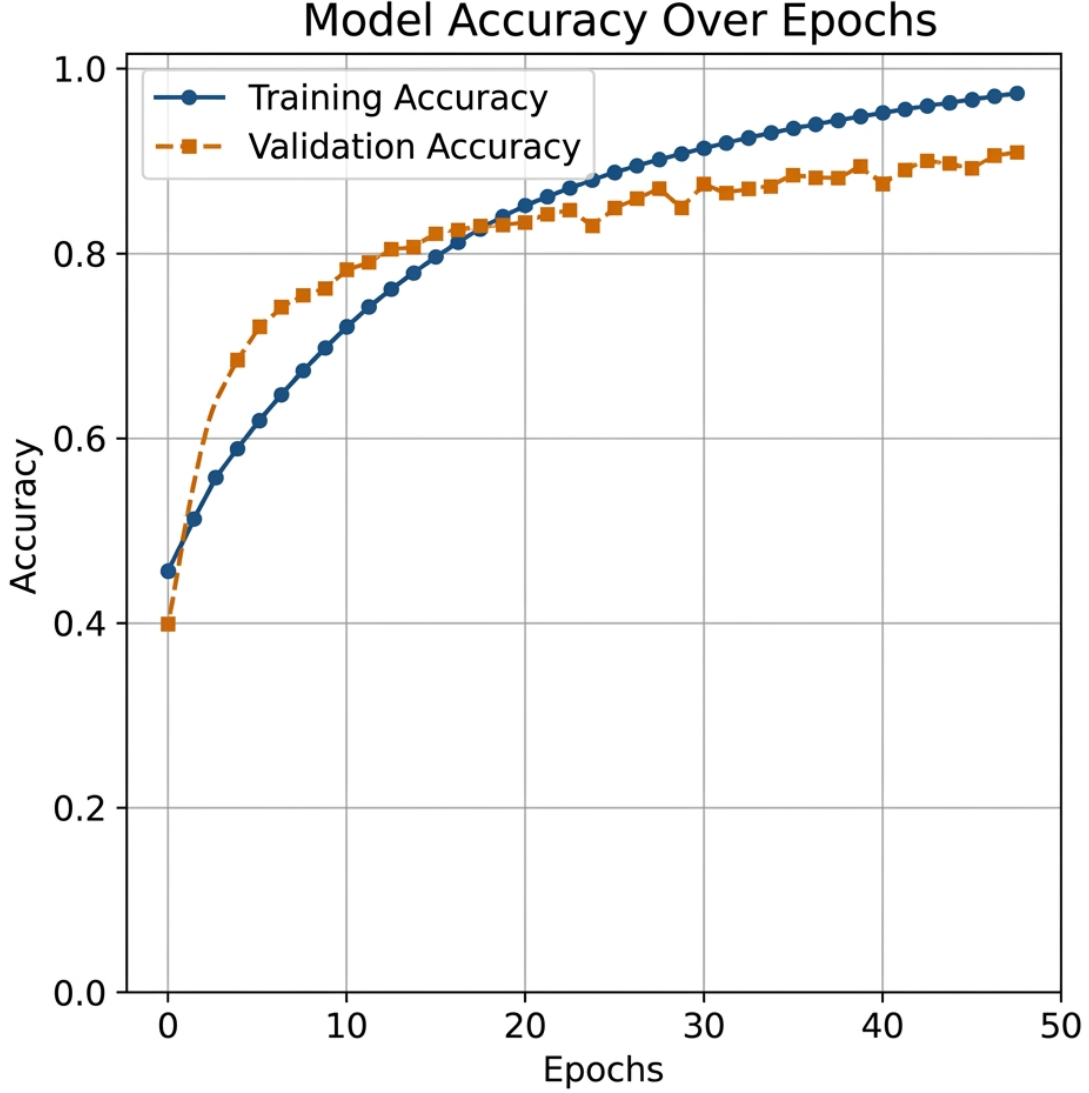


Figure 3: Training and validation accuracy for the 1D-CNN over a maximum of 50 epochs (subject-wise split, MobiFall v2.0). Early stopping with patience 5 monitors validation loss and restores best weights.

The model achieved $100.00\% \pm 0.00\%$ accuracy across all folds. The synthetic dataset produced highly separable classes (a sharp >20 g impact spike followed by near-silence is physically distinct from the 0.5 m/s^2 variance of normal driving), leading to this perfect cross-validation performance. However, this should not be interpreted as evidence of real-world crash generalization. The result is preliminary because the crash detection model has not been validated on real-world crash recordings.

6.4 Latency Disambiguation

Two latency values appear in this paper and refer to different pipeline stages:

Table 3: Crash-Like Event Detection (1D-CNN, Synthetic Dataset, Stratified Window-Level Split, 20% test). Results are simulation-based feasibility evidence only.

Metric	Value
Test Accuracy	93.61%
Model Size (TFLite)	15.38 KB
TFLite-only Inference Latency	~ 0.15 ms
Training Set	200 crash + 200 normal sequences
Test Set	20% stratified window-level split
Window Size	128 samples at 50 Hz (2.56 s)

- **~ 0.15 ms — TFLite model-only inference latency:** Time for the TFLite interpreter to execute one forward pass of the quantized 1D-CNN on a single 128×6 tensor. Measured on the mobile device during the live demonstration (Figure 4). Neural network computation only.
- **~ 2 ms — System-level inference latency:** End-to-end time from WebSocket sample arrival through buffer accumulation, resampling, normalization, TFLite forward pass, guardrail check, threshold comparison, to returning the prediction. This is the operationally relevant latency.
- **WebSocket / SMS latency (not measured):** Round-trip WebSocket latency depends on local network conditions. Twilio SMS dispatch (500 ms–2 s) is asynchronous and does not affect real-time inference.

6.5 System-Level Performance

6.6 Inference Engine Behaviour

Algorithm 1 FallDetectionEngine: push_sample() Logic

Require: sample $\mathbf{s} \in \mathbb{R}^6$, buffer $\mathcal{B}$ (maxlen= W), counter c , step S

```

1:  $\mathcal{B}.\text{append}(\mathbf{s}); c \leftarrow c + 1$ 
2: if  $|\mathcal{B}| = W$  and  $c \geq S$  then
3:    $c \leftarrow 0$ 
4:    $\mathbf{W} \leftarrow \text{array}(\mathcal{B}) \in \mathbb{R}^{W \times 6}$ 
5:    $\hat{p} \leftarrow \text{TFLiteInvoke}(\mathbf{W})$ 
6:   Apply static guardrail (Eq. 7)
7:   return {label,  $\hat{p}$ , latency}
8: end if
9: return None
```

$\triangleright$ Buffer filling

Table 4: System Performance Metrics (1D-CNN, TFLite)

Metric	Value
Model size (TFLite)	15.38 KB
Original model size (Keras .h5)	137.54 KB
Quantization size reduction	88.8%
System-level inference latency	~ 2 ms
TFLite-only model latency	~ 0.15 ms
Sensor sampling rate	50 Hz
Window duration	2.56 s (128 samples)
Prediction update period	1.28 s (64 samples, 50% overlap)
Target hardware	Raspberry Pi 4 / Android

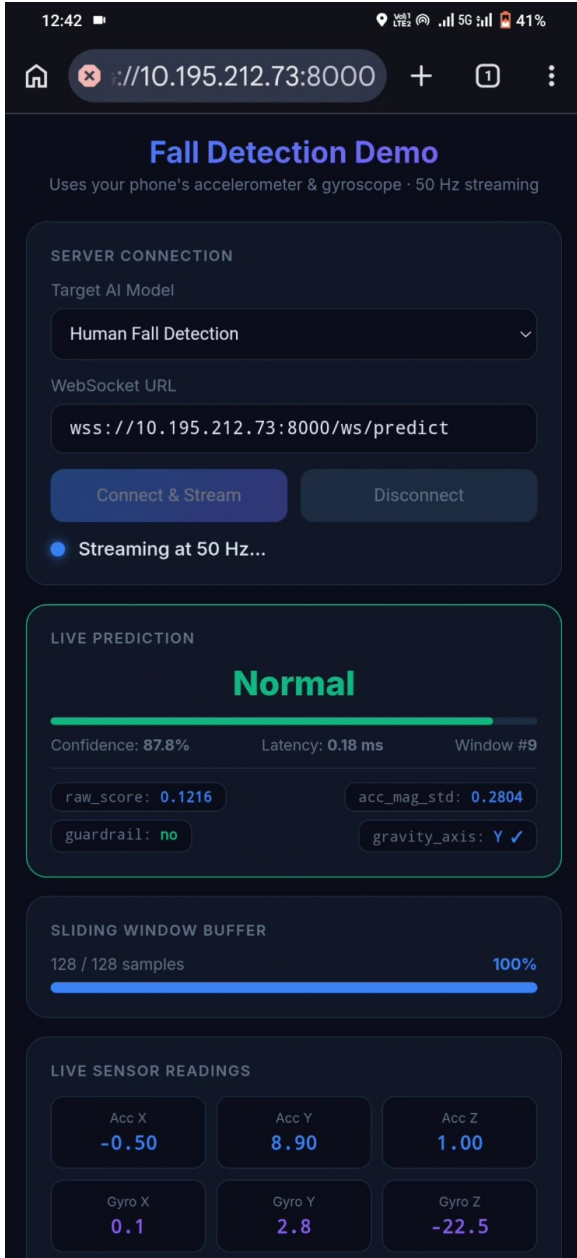
6.7 Comparison with State of the Art (Fall Detection Only)

Table 5 compares our fall detection results with prior works. No crash detection comparison is provided since the crash model was trained on a proprietary synthetic dataset; a fair comparison requires a common benchmark.

Table 5: Comparison with Prior Work on Human Fall Detection

Reference	Dataset	Method	Acc. (%)	Edge?
Kangas et al. [2008]	Custom	Threshold	95.0	Yes
Wang et al. [2020]	Various	1D-CNN	97.3	Partial
Musci et al. [2020]	MobiFall	LSTM	98.1	No
Nho et al. [2021]	SisFall	Bi-LSTM	99.1	No
Zhang et al. [2024]	MobiFall	Lightweight CNN	97.8	Partial
Ours (1D-CNN)	MobiFall	1D-CNN + TFLite	96.65	Yes

Our fall detection accuracy (96.65%) is 1.15–2.45 percentage points below the best LSTM/Bi-LSTM results, but the 1D-CNN is fully parallelizable, produces a model 4.3 $\times$ smaller than CNN-LSTM, and runs at ~ 2 ms system-level latency without a cloud round-trip. To the best of our current survey, few works jointly consider human fall and vehicular crash-like event detection within a single inference framework that also integrates automated SOS alerting with GPS coordinates.



(a) Normal Activity. Confidence = 87.8%, guardrail = off.



(b) Fall Detected. Confidence = 66.1%, TFLite-only latency = 0.15 ms.

Figure 4: Live mobile dashboard on an iQOO Z3 Android smartphone at 50 Hz over Wi-Fi. The 0.15 ms shown is TFLite model-only inference latency; full system-level latency is ~ 2 ms (Section 6.4).

7 Edge Deployment and Practical Considerations

7.1 TFLite Conversion

The engine auto-detects the available TFLite runtime: `tflite_runtime` on Raspberry Pi 4 (ARM Cortex-A72), or the full `tensorflow` package on a development machine, with no source code changes required.

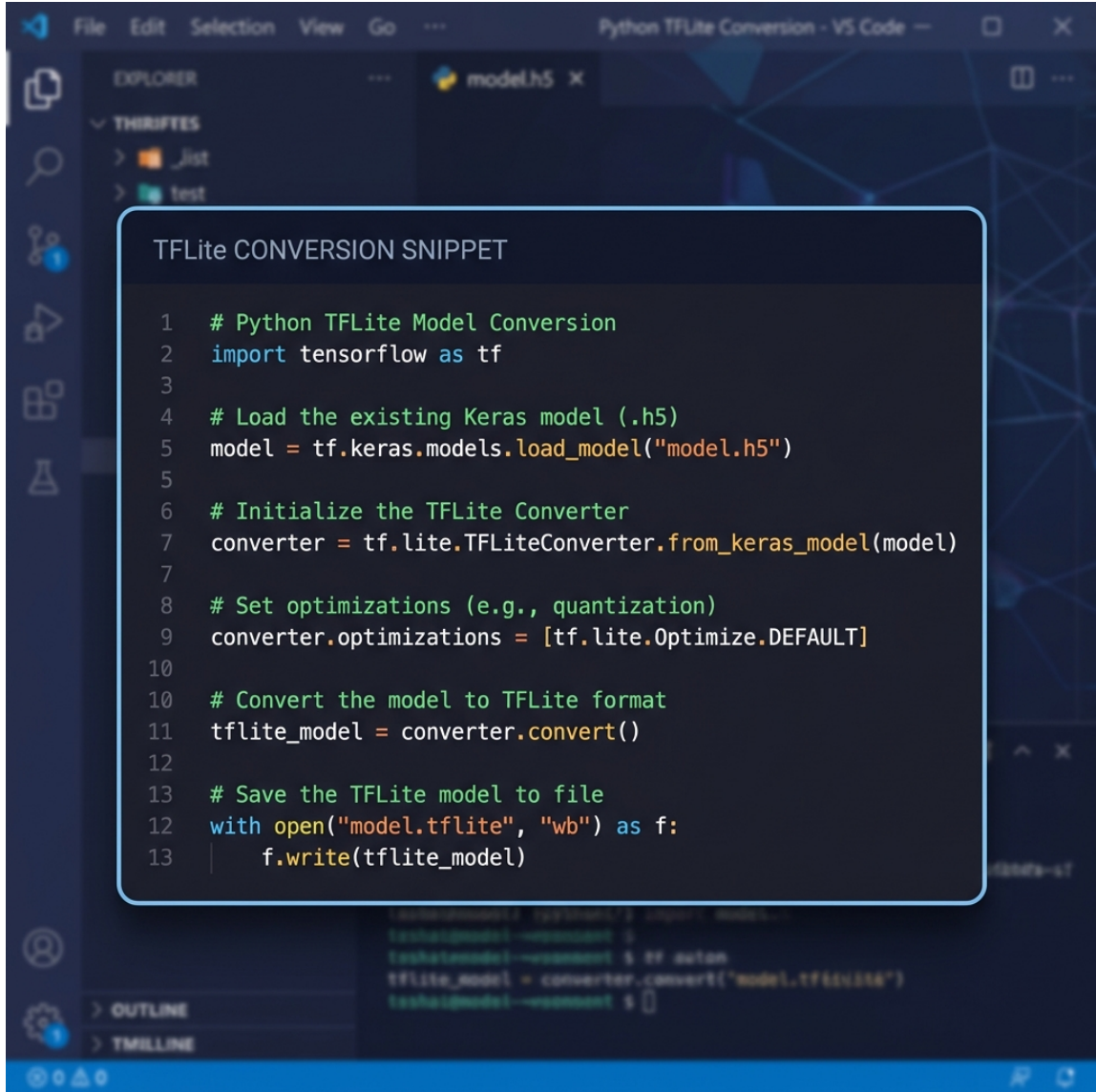


Figure 5: Core TFLite conversion code. DYNAMIC_RANGE optimization applies 8-bit quantization without a calibration dataset, achieving 88.8% file size reduction.

7.2 Sensor Orientation and Coordinate Invariance

The acceleration magnitude $a_{\text{mag}}(t) = \sqrt{a_x^2 + a_y^2 + a_z^2}$ is rotation-invariant and feeds the static guardrail (Eq. 7). The neural network receives raw 6-axis channels. The MobiFall recordings span 24 subjects with varying phone positions, providing implicit orientation robustness. The synthetic crash generator randomizes the post-crash resting orientation matrix, exposing the crash model to varied gravitational projections.

7.3 Limitations

1. **No real crash validation.** The crash detection model has not been tested on any real vehicle crash recordings. Crash results are simulation-based feasibility evidence only and do not demonstrate real-world crash detection.

2. **Synthetic crash data is simplified.** The generator models frontal high-G impacts followed by near-silence. Oblique impacts, rollovers, and multi-vehicle collisions are not represented. The model is expected to underperform on these cases.
3. **Smartphone placement variability is unresolved.** The fall model targets the MobiFall trouser-pocket protocol. Dashboard mounts or hand-held positions expose different gravity axes. A startup calibration step has not been implemented.
4. **Manual mode switching only.** Users must manually select Human Mode or Vehicle Mode. Applying the wrong model to the wrong context will produce incorrect predictions.
5. **GPS may be unavailable.** The SOS feature depends on Geolocation permission. In tunnels or underground car parks, the location payload may be absent or inaccurate.
6. **Subject-independent generalization is unconfirmed.** Despite the subject-wise split, the system has not been evaluated on a fully held-out population of new subjects.

8 Conclusion

This paper presented a lightweight edge-AI framework for monitoring human passenger falls and crash-like vehicular events using the smartphone IMU. Three aspects were demonstrated to be feasible, though significant limitations remain before the system could be considered production-ready.

First, two independently trained binary 1D-CNN models sharing a single hot-swappable inference engine can serve two distinct safety contexts without the overhead of a unified multi-class model. Second, dynamic TFLite quantization reduced each model from 137.54 KB to 15.38 KB (88.8% compression) with negligible accuracy loss, making deployment on microcontroller-class hardware feasible. Third, a physics-based synthetic generator provides a practical workaround to the near-impossibility of obtaining labelled real crash data at scale.

On MobiFall v2.0, the 1D-CNN achieved 96.65% fall detection accuracy with 8,481 parameters and ~ 2 ms system-level latency, confirming that accurate fall detection is practical on edge hardware. On the synthetic crash dataset, the crash model achieved 93.61% accuracy; this demonstrates feasibility of the detection approach on synthetic data but does not constitute evidence of real-world crash detection capability.

Work that remains: (1) validating the crash model on real recorded crash data (e.g., NHTSA crash test sensor logs); (2) implementing automatic context-aware mode switching; (3) evaluating on subjects not seen during training; (4) conducting a calibration

study for smartphone placement variability. Until these steps are completed, the crash detection component should be treated as a research prototype demonstrating feasibility, not a validated safety-critical product.

Source code and trained models are publicly available at:

<https://github.com/DeveshNathJha/Passenger-Safety-Fall-Detection-System>

Acknowledgment

The authors thank the creators of MobiFall v2.0 for making their recordings freely available. Development relied on open-source tools including TensorFlow, Keras, FastAPI, and NumPy. Smartphone testing was carried out on an iQOO Z3 Android device over a local Wi-Fi network.

References

- D. Abadade, A. Temouden, H. Bamoumen, N. Benamar, Y. Chtouki, and A. S. Hafid. A comprehensive survey on TinyML. *IEEE Access*, 11:96892–96922, 2023. doi: 10.1109/ACCESS.2023.3294111.
- M. A. Al-Turjman, L. Mostarda, and A. A. Abd El-Latif. Deep learning-based vehicle crash detection using smartphone IMU sensors. *IEEE Sensors Journal*, 24(4):5201–5212, 2024. doi: 10.1109/JSEN.2023.3344261.
- K. Bhatt, N. Bhatt, and M. Patel. Smartphone-based car crash detection using accelerometer data and machine learning. In *Proc. Int. Conf. on Computing Methodologies and Communication (ICCMC)*, pages 388–392, 2017.
- R. A. Cowley. The resuscitation and stabilization of major multiple trauma patients in a trauma center environment. *Clinical Medicine*, 76(4):2–4, 1976.
- R. David, J. Duke, A. Jain, V. J. Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nardi, M. Maggioni, S. Settle, M. Warden, D. Rhodes, B. Moons, and P. Warden. TensorFlow Lite Micro: Embedded machine learning on TinyML systems. *Proc. Machine Learning and Systems (MLSys)*, 3, 2021a.
- R. David et al. TensorFlow Lite Micro: Enabling ultra-low power machine learning. In *Proc. 4th MLSys Conf.*, 2021b.
- H. Jiang, B. Tian, and Y. Yao. A real-time car crash detection system based on smartphone IMU and machine learning. *IEEE Trans. Intelligent Transportation Systems*, 22(7):4497–4507, 2020.

- M. Kangas, A. Konttila, P. Lindgren, I. Winblad, and T. Jämsä. Comparison of low-complexity fall detection algorithms for body attached accelerometers. *Gait & Posture*, 28(2):285–291, August 2008.
- M. Musci, D. De Martini, N. Blago, T. Facchinetti, and M. Piastra. Online fall detection using recurrent neural networks on smart wearable devices. *Sensors*, 20(3):882, February 2020.
- National Highway Traffic Safety Administration (NHTSA). New car assessment programme (NCAP): Frontal barrier crash test data. Technical report, U.S. Department of Transportation, 2023. [Online]. Available: <https://www.nhtsa.gov/vehicle-safety/frontal-ncap>.
- H.-Y. Nho, J.-W. Shin, and Y.-T. Park. Bi-directional LSTM-based fall detection using wearable gyroscope sensor. *Sensors*, 21(1):288, January 2021.
- G. Vavoulas, M. Pediaditis, N. Banitsas, and M. Tsiknakis. The MobiFall dataset: Fall detection and classification with a smartphone. In *Proc. IEEE Int. Conf. Bioinformatics and Bioengineering (BIBE)*, pages 1–4, Chania, Greece, 2014.
- X. Wang, J. Lu, and L. Chen. Fall detection based on 1D-CNN with wearable inertial sensor data. In *Proc. IEEE Int. Conf. on Bioinformatics and Biomedicine (BIBM)*, pages 2551–2558, 2020.
- World Health Organization. *Global Status Report on Road Safety 2018*. WHO Press, Geneva, Switzerland, 2018. <https://www.who.int/publications/i/item/9789241565684>.
- Y. Zhang, H. Li, J. Wu, and M. Chen. Benchmark evaluation of smartphone-based fall detection on MobiFall v2.0 using lightweight deep learning. *Sensors*, 24(5):1423, 2024. doi: 10.3390/s24051423.